

# Documentación Académica y Técnica

## Carátula

|                            |                                            |
|----------------------------|--------------------------------------------|
| <b>Institución</b>         | Universidad Tecnológica Nacional           |
| <b>Carrera</b>             | Tecnicatura en programacion                |
| <b>Materia</b>             | Programación 1                             |
| <b>Trabajo</b>             | Trabajo Práctico Integrador (TPI)          |
| <b>Título del proyecto</b> | Gestión para los datos de paises en python |
| <b>Integrantes</b>         | Mercado Gabriel Emiliano                   |
| <b>Fecha de entrega</b>    | 16 JUN 2026                                |

## Índice

1. Introducción **pág. 2**
2. Marco teórico **pág 3**
  - 2.1 Listas
  - 2.2 Diccionarios
  - 2.3 Funciones
  - 2.4 Condicionales
  - 2.5 Ordenamientos
  - 2.6 Estadísticas básicas
  - 2.7 Archivos CSV
3. Decisiones técnicas y arquitectura **pág 9**

4. Dificultades y conclusiones **pág 13**
5. Bibliografía y webgrafía **pág 13**
6. Enlaces del proyecto **pág 14**

# 1. Introducción

Este documento acompaña la entrega del Trabajo Práctico Integrador correspondiente a la materia Programación 1. El objetivo del proyecto es desarrollar una aplicación de consola en Python para gestionar información de países a partir de un archivo CSV.

La app implementa las funcionalidades exigidas por la consigna: alta y actualización de registros, búsqueda por nombre, filtrado por continente y por rangos numéricos, ordenamiento ascendente/descendente, y calcula las estadísticas básicas. Además, incluye validación de entradas, manejo de errores y persistencia de datos en disco.

La estructura central del programa es una **lista de diccionarios** en memoria, donde cada diccionario representa un país con los campos **nombre**, **población**, **superficie** y **continente**.

## Archivos principales del proyecto:

| Archivo                       | Rol                                                 |
|-------------------------------|-----------------------------------------------------|
| <code>tp-integrador.py</code> | Aplicación principal con menú de consola            |
| <code>países.csv</code>       | Dataset persistente                                 |
| <code>generar_datos.py</code> | Script para regenerar el CSV desde fuentes abiertas |
| <code>FUENTES.md</code>       | Documentación de fuentes de datos                   |
| <code>README.md</code>        | el readme principal del proyecto                    |

## 2. Marco teórico

El marco teórico describe los conceptos de programación aplicados en el desarrollo del sistema. El código se apoya en fuentes bibliográficas y documentación oficial.

### 2.1 Listas

#### Concepto

Una **lista** (`list`) es una estructura de datos ordenada y mutable que permite almacenar una colección de elementos (Python Software Foundation, 2026a). En Python, las listas se definen con corchetes y admiten operaciones como agregar elementos, recorrerlos e indexarlos por posición.

#### Aplicación en el proyecto

El conjunto de países se almacena en una lista llamada **countries**. Al iniciar el programa, `load_countries()` lee el CSV y construye la lista:

```
countries.append(  
    {  
        "nombre": row["nombre"].strip(),  
        "poblacion": int(row["poblacion"]),  
        "superficie": int(row["superficie"]),  
        "continente": row["continente"].strip(),  
    }  
)
```

Durante la ejecución, todas las operaciones del menú trabajan sobre esta lista en memoria. Cuando el usuario agrega o actualiza un país, se modifica la lista y luego se persiste con `save_countries()`.

#### Operaciones relevantes

- `append()` – incorporar un país nuevo
- `len()` — conocer la cantidad de registros cargados
- `sorted()` — obtener una copia ordenada sin alterar la lista original
- **Comprensión de listas** — filtrar resultados en una sola expresión:

```
results = [c for c in countries if c["continente"] == continent]
```

Esta construcción es equivalente a un bucle **for** con condición, pero más concisa para generar subconjuntos.

---

## 2.2 Diccionarios

### Concepto

Un **diccionario** (**dict**) almacena pares **key-value**, lo que permite acceder a cada dato por un identificador simbólico en lugar de por posición (Python Software Foundation, 2026a). Es la estructura más adecuada para representar entidades con atributos nombrados.

### Aplicación en el proyecto

Cada país se modela como un diccionario de cuatro claves fijas:

```
{  
    "nombre": "Argentina",  
    "poblacion": 45696159,  
    "superficie": 2780400,  
    "continente": "América",  
}
```

Las claves coinciden con las columnas del CSV, lo que facilita la lectura y escritura con el módulo csv.

También se utilizan diccionarios auxiliares:

- `criterion_map()` — traduce la opción del menú al campo de ordenamiento
- `count_by_continent()` — acumula cuántos países hay por continente
- `CONTINENT_MAP` (en el archivo `generar_datos.py`) - mapea regiones ONU al español

### Ventajas:

Usar diccionarios mejora la legibilidad: `country["poblacion"]` es más claro que acceder por índice numérico, y el código se mantiene alineado con el formato del archivo de datos.

---

## 2.3 Funciones

### Concepto

Una **función** es un bloque de código reutilizable que cumple una tarea específica. En el diseño del proyecto se siguió el principio de **una función, una responsabilidad**, recomendado en la guía de estilo y buenas prácticas de Python (Python Software Foundation, 2026b).

## Aplicación en el proyecto

El código se organizó en grupos funcionales:

| Grupo        | Funciones                                                 | Responsabilidad                 |
|--------------|-----------------------------------------------------------|---------------------------------|
| Persistencia | load_countries, save_countries                            | Lectura y escritura del CSV     |
| Entrada      | read_text, read_positive_int, read_continent, read_range  | Validación de datos del usuario |
| Consulta     | find_country_index, search_country                        | Búsqueda de países              |
| Filtrado     | filter_by_continent, filter_by_population, filter_by_area | Subconjuntos por criterio       |
| Presentación | display_countries, show_menu                              | Salida formateada en consola    |
| Análisis     | show_statistics, sort_countries                           | Estadísticas y ordenamiento     |
| Control      | main                                                      | Bucle principal del menú        |

## Beneficios observados

- **Reutilización:** read\_positive\_int() se usa tanto al agregar como al actualizar países
- **Mantenimiento:** un cambio en la validación se realiza en un solo lugar
- **Claridad:** main() actúa como orquestador sin contener toda la lógica de negocio

## Funciones anónimas “lambda”

Para el ordenamiento se emplean funciones pequeñas sin nombre que indican a sorted() qué valor comparar:

```
sort_key = (  
    (lambda c : {__getitem__} : c[field].casefold())  
    if field == "nombre"  
    else (lambda c : {__getitem__} : c[field])  
)
```

Lambda permite definir funciones de una sola expresión; en este caso, extraer el criterio de comparación de cada país (Python Software Foundation, 2026a).

## 2.4 Condicionales

### Concepto

Las estructuras **condicionales** (if, elif, else) permiten ejecutar código solo cuando se cumple una condición lógica (Python Software Foundation, 2026c). Son fundamentales para validar entradas, controlar el flujo del menú y filtrar datos.

### Aplicación en el proyecto

#### Validación de entrada del usuario:

```
if number <= 0:
    print("Error: debe ingresar un numero entero positivo.")
    continue
```

#### Control del menú principal:

```
if option == "1":
    add_country(countries)
elif option == "2":
    update_country(countries)

else:
    print("Error: opcion no valida.")
```

#### Manejo de errores al cargar el CSV:

```
try:
    countries: list[dict] = load_countries()
except (FileNotFoundError, ValueError) as error:
    print(f"Error al iniciar: {error}")
    return
```

El uso combinado de condicionales y try / except evita que el programa se explote todo si encuentra un error, formatos inválidos o datos incorrectos.

---

## 2.5 Ordenamientos

### Concepto

Ordenar un conjunto de datos implica disponer los elementos según un criterio (nombre, población o superficie) y en un orden (ascendente o descendente). Python te da la función **built-in sorted()**, que implementa el algoritmo *Timsort*, eficiente para listas parcialmente ordenadas (Python Software Foundation, 2026a).

Parámetros utilizados

| Parámetro | Función en el proyecto                                     |
|-----------|------------------------------------------------------------|
| Iterable  | Lista de países (countries)                                |
| key       | Función que extrae el valor a comparar                     |
| reverse   | <b>True</b> para descendente, <b>False</b> para ascendente |

Criterios implementados

- **Por nombre:** se usa `.casefold()` para ignorar diferencias de mayúsculas/minúsculas
- **Por población o superficie:** se comparan directamente los valores enteros

```
# Esto crea una nueva lista dict usando sort (ordenar) con el "criterio" elegido como key
sorted_countries = sorted(countries, key=sort_key, reverse=descending)
```

`sorted()` devuelve una **nueva lista** sin modificar la original, lo cual es adecuado para mostrar resultados temporales al usuario.

## 2.6 Estadísticas básicas

Concepto

Las **estadísticas descriptivas básicas** resumen un conjunto numérico mediante medidas como valores extremos (máximo y mínimo), tendencia central (promedio) y conteos por categoría (Newman, 2013). En este proyecto se aplican sobre los campos `poblacion` y `superficie`

Medidas implementadas

| Medida | Descripción          | Código                                                    |
|--------|----------------------|-----------------------------------------------------------|
| Máximo | País con mayor valor | <code>max(countries, key=lambda c: c["poblacion"])</code> |
| Mínimo | País con menor valor | <code>min(countries, key=lambda c: c["poblacion"])</code> |

|          |                            |                                        |
|----------|----------------------------|----------------------------------------|
| Promedio | Suma dividida por cantidad | <code>sum(...) / len(countries)</code> |
| Conteo   | Países por continente      | Diccionario acumulador                 |

Ejemplo: promedio de población

```
avg_population = sum(c["poblacion"] for c in countries) / len(countries)
```

Se recorre la lista sumando poblaciones y luego se divide por el total de países. El uso de `max()` y `min()` con `key` permite obtener el **registro completo** del país extremo, no solo el número, lo que mejora la utilidad de la salida para el usuario.

## 2.7 Archivos CSV

Concepto

**CSV** (*Comma-Separated Values*) es un formato de texto plano para representar datos tabulares, donde cada línea es un registro y los campos se separan por comas (Shafranovich, 2005; Python Software Foundation, 2026d). Es ampliamente utilizado por su simplicidad, interoperabilidad con hojas de cálculo y facilidad de procesamiento en Python.

Formato del proyecto

```
nombre, poblacion, superficie, continente
Argentina, 45696159, 2780400, América
Japón, 123975371, 377969, Asia
```

Lectura y escritura

Python incluye el módulo `csv` con clases especializadas:

- `csv.DictReader` — lee cada fila como diccionario usando la primera línea como encabezado
- `csv.DictWriter` — escribe diccionarios respetando un conjunto de columnas definido

```
reader = csv.DictReader(file)
```



```
with CSV_FILE.open( mode: "w", newline="", encoding="utf-8") as file:|
    writer = csv.DictWriter(file, fieldnames=FIELDS)
    writer.writeheader()
    writer.writerows(countries)
```

#### Validaciones implementadas

1. Que exista el archivo
2. Columnas esperadas (nombre, poblacion, superficie, continente)
3. Conversión de valores numéricos a int
4. Mensajes de error con número de línea cuando corresponde

#### Origen de los datos

El CSV se genera con generar\_datos.py combinando:

- **Banco Mundial** — indicadores **SP.POP.TOTL** (población) y **AG.SRF.TOTL.K2** (superficie)
- **mledoze/countries** — nombres en español y región ONU

Los registros se unen por código **ISO-3166 alpha-3**. La API del Banco Mundial devuelve tanto países como agregados regionales; el script descarta entradas sin valor numérico y utiliza mledoze como lista maestra de países reales. En otras palabras agarra la lista de código de países del repo de mledoze y después busca esas keys en la lista del banco mundial y extrae los datos para cada uno de esos registros para armar el CSV

## 3. Decisiones técnicas y arquitectura

### 3.1 Decisiones de diseño

| Decisión                         | Motivo de la elección                                           |
|----------------------------------|-----------------------------------------------------------------|
| Lista de diccionarios en memoria | Cumple la consigna con estructuras del curso; menor complejidad |
| CSV como persistencia            | Formato exigido por la consigna; soporte nativo en Python       |
| Menú de consola                  | Consigna orientada a consola; no requiere librerías externas    |

|                                                                                                                                                           |                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
|                                                                                                                                                           | aunque idealmente esto seria consumido en forma de API y los datos de una DB |
| Nombres de funciones en inglés, mensajes en español ya que usualmente los reclutadores prefieren inglés y demuestra capacidad para trabajar internacional | Claridad de código + interfaz en español para el usuario final               |
| Fuentes abiertas verificables                                                                                                                             | Reproducibilidad y todo citable para el informe académico                    |
| sorted() con key                                                                                                                                          | Menor riesgo de error; función estándar y eficiente                          |

## 3.2 Arquitectura general

Menú repetitivo con un **while** ya que funcionó bien para los menú de inventario usados en los parciales anteriores. Pero principalmente se debe a que es una app corriendo en la terminal, idealmente esto sería una API que consume distintos servicios donde hay una capa de logica interna de negocio y expone diferentes metodos que se invocan desde un controlador con un endpoint, esto no se guardaría en un CSV sino en una DB posiblemente SQL y los datos de la API serían consumidos por una app de Front-end donde se puede filtrar y visualizar con facilidad.

## 3.3 Flujo principal de operaciones

Inicio: try / except -> se intenta cargar los datos del csv o tira error si falla

menu **WHILE TRUE**

-> mostrar menu

-> leer opcion

IF opcion = 1 -> agregar un pais

elif opcion = 2 -> actualizar pais

elif opcion = 3 -> buscar un pais

elif opcion = 4 -> filtrar por continente

elif opcion = 5 -> filtrar por poblacion

elif opcion = 6 -> filtrar por superficie

elif opcion = 7 -> filtrar paises por criterio particular -> nombre | poblacion | superficie

elif opcion = 8 -> mostrar estadisticas

elif opcion = 9 -> mostrar lista de todos los paises

elif opcion = 10 -> mostrar las fuentes (lee el texto del archivo md)

elif opcion = 0 -> salir del programa while **BREAK**

else -> error opcion no valida.

### 3.6 Capturas de pantalla

The screenshot shows a Visual Studio Code editor with a project named "tec-programacion-1". The file explorer on the left shows the project structure, including a folder "programacion-1" containing a file "tp-integrador.py".

The main editor displays the contents of "tp-integrador.py":`179 def main_menu() -> None:
180 print("\n Por las fuentes de los datos")
181 print("0. salir")
182 print("=====")
183
184 def menu() -> None:
185 """Punto de entrada para la aplicacion"""
186 try:
187 countries = load_countries()
188 except FileNotFoundError as error:
189 print(f'Error al iniciar: {error}')
190 return
191
192 print(f'Se cargaron {len(countries)} países desde ({CSV_FILE_NAME})')
193
194 while True:
195 show_menu()
196 option = input('Seleccione una opción: ').strip()
197
198 if option == "1":
199 add_country(countries)
200 elif option == "2":
201 update_country(countries)
202 elif option == "3":
203 search_country(countries)
204 elif option == "4":
205 filter_by_continent(countries)
206 elif option == "5":
207 filter_by_population(countries)
208 else:
209 continue`

The Run and Debug panel at the bottom shows the execution of the script. The command used is:

```
/usr/bin/python3 /Users/patrickdes/personal/tec-programacion/programacion-1/trabajo-practico-integrador/tp-integrador.py
```

The output of the program is:

```
Se cargaron 215 países desde países.csv.
===== LISTA DE PAISES =====
1. Agregar país
2. Actualizar población y superficie
3. Borrar país
4. Filtrar por continente
5. Filtrar por rango de población
6. Filtrar por rango de superficie
7. Ordenar países
8. Mostrar los estadísticas
9. Mostrar lista de todos los países
10. Ver las Fuentes de los datos
0. Salir
Seleccione una opción:
```

=====

Seleccione una opcion: 3

--- Buscar país ---

- 1. Coincidencia exacta
- 2. Coincidencia parcial

Opcion: 2

Nombre a buscar: arg

--- Resultados de búsqueda (2) ---

| Nombre    | Poblacion  | Superficie | Continente |
|-----------|------------|------------|------------|
| Argelia   | 46,814,308 | 2,381,740  | África     |
| Argentina | 45,696,159 | 2,780,400  | América    |

=====

Seleccione una opcion: 5

--- Filtrar por rango de poblacion ---

Poblacion minima: 1408975000

Poblacion maxima: 1408975000

--- Filtro por poblacion (1) ---

| Nombre | Poblacion     | Superficie | Continente |
|--------|---------------|------------|------------|
| China  | 1,408,975,000 | 9,562,910  | Asia       |

=====

Seleccione una opcion: 0

Saliendo del programa.

Process finished with exit code 0

.

## 4. Dificultades y conclusiones

Fue bastante difícil encontrar una fuente de datos que mas o menos tenga los datos en un formato que pueda entender y ver como extraerlos y transformarlos para poder trabajarlos. Tomó algo de tiempo de investigación, y combinarlos con datos de otra fuente fue una tarea relativamente difícil como hacer un JOIN de tablas en SQL (parecido). También fue difícil usar los formatos en f-strings (`<28`, `>14`,) y entender como funcionan ya que normalmente esto no lo uso mucho porque no necesito mostrar las cosas de forma tan prolija o alineada en la terminal. Pero sobre todas las cosas la mas difícil fue escribir toda esta documentación redundante en PDF para cumplir requerimientos académicos, normalmente este proceso es distinto a la hora de documentar software.

## 5. Bibliografía y webgrafía

### Documentación oficial

- Python Software Foundation. (2026a). *The Python Tutorial — More on Lists, Dictionaries, Sorting, and Lambda*. <https://docs.python.org/3/tutorial/datastructures.html>
- Python Software Foundation. (2026b). *PEP 8 – Style Guide for Python Code*. <https://peps.python.org/pep-0008/>
- Python Software Foundation. (2026c). *The Python Tutorial — if Statements*. <https://docs.python.org/3/tutorial/controlflow.html>
- Python Software Foundation. (2026d). *csv — CSV File Reading and Writing*. <https://docs.python.org/3/library/csv.html>

### Fuentes de datos

- Banco Mundial. (2026). *World Development Indicators: Population, total (SP.POP.TOTL)*. <https://data.worldbank.org/indicator/SP.POP.TOTL>
- Banco Mundial. (2026). *World Development Indicators: Surface area (sq. km) (AG.SRF.TOTL.K2)*. <https://data.worldbank.org/indicator/AG.SRF.TOTL.K2>
- Banco Mundial. (2026). *API Documentation*. <https://datahelpdesk.worldbank.org/knowledgebase/articles/889392>
- Ledoze, M. (2026). *Countries of the world [Dataset]*. GitHub. <https://github.com/mledoze/countries>

### Referencias complementarias

- Newman, M. (2013). *Data Analysis for Politics and Policy*. Englewood Cliffs, NJ: Prentice-Hall. (Estadística descriptiva básica)
- Shafranovich, M. (2005). *Common Format and MIME Type for CSV Files* (RFC 4180). <https://www.rfc-editor.org/rfc/rfc4180>
- W3Schools. (2026). *Python Lists, Dictionaries and Functions*. <https://www.w3schools.com/python/>

## 6. Enlaces del proyecto

| Recurso            | Enlace                                                                                                                                                                                                                                |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repositorio GitHub | <a href="https://github.com/mercadogabriel91/tec-programacion/tree/master/programacion-1/trabajo-practico-integrador">https://github.com/mercadogabriel91/tec-programacion/tree/master/programacion-1/trabajo-practico-integrador</a> |
| Video demostración | <a href="https://youtu.be/-xfSo7MxmzY">https://youtu.be/-xfSo7MxmzY</a>                                                                                                                                                               |